บทที่ 3: การจัดเก็บและการค้นคืนข้อมูล

(สรุปเป็นภาษาไทยจากฉบับที่ 1 ใช้เวลาฟังประมาณ 18 นาที ไม่มีคำว่า [หยุด] แล้ว)

ตอนที่ 1: บทนำ

สวัสดีครับ ยินดีต้อนรับสู่บทสรุปหนังสือ Designing Data-Intensive Applications บทที่สาม ว่าด้วยการจัดเก็บและการค้นคืนข้อมูล

ฐานข้อมูลมีหน้าที่พื้นฐานแค่สองอย่าง คือ เมื่อเราส่งข้อมูลให้ มันต้องเก็บไว้ และเมื่อเราขอข้อมูลคืน มันต้องหามาให้ได้

ในบทที่แล้ว เราดูเรื่องนี้จากมุมของนักพัฒนา ว่าจะส่งข้อมูลเข้าไปและขอข้อมูลกลับมาในรูปแบบไหน บทนี้จะเปลี่ยนไปดูจากมุมของฐานข้อมูลเอง ว่ามันเก็บข้อมูลลงดิสก์อย่างไร และหาข้อมูลเจอได้อย่างไร

บางคนอาจสงสัยว่า นักพัฒนาต้องรู้เรื่องนี้ด้วยหรือ เพราะเราคงไม่ได้เขียนฐานข้อมูลเอง คำตอบคือ ต้องรู้ เพราะเราต้องเลือกฐานข้อมูลให้เหมาะกับงาน และต้องปรับแต่งให้ทำงานได้ดี ถ้าไม่เข้าใจว่าข้างในทำงานอย่างไร ก็เลือกและปรับแต่งได้ยาก

บทนี้แบ่งเป็นสองส่วนใหญ่ ส่วนแรกคือระบบจัดเก็บสำหรับงานธุรกรรมทั่วไป ส่วนที่สองคือระบบจัดเก็บสำหรับงานวิเคราะห์ข้อมูล ทั้งสองแบบต้องออกแบบต่างกันมากครับ

ตอนที่ 2: ฐานข้อมูลที่ง่ายที่สุดในโลก

ผู้เขียนเริ่มด้วยตัวอย่างที่ง่ายมาก คือฐานข้อมูลที่มีแค่สองคำสั่ง คำสั่งแรกใช้บันทึกข้อมูล โดยเอาคีย์กับค่าไปต่อท้ายไฟล์ คำสั่งที่สองใช้ค้นหาข้อมูล โดยอ่านไฟล์ทั้งไฟล์ แล้วเอาค่าล่าสุดของคีย์นั้นมา

ถ้าแก้ค่าของคีย์เดิม ก็แค่เขียนบรรทัดใหม่ต่อท้าย ไม่ต้องลบของเก่า การเขียนต่อท้ายไปเรื่อย ๆ แบบนี้เรียกว่า log หรือบันทึกแบบต่อท้าย

การเขียนแบบนี้เร็วมาก เพราะแค่ต่อท้ายไฟล์ ฐานข้อมูลจริงหลายตัวก็ใช้แนวคิดนี้อยู่ข้างใน

แต่การอ่านช้ามาก เพราะต้องอ่านทั้งไฟล์ทุกครั้ง ถ้าข้อมูลเพิ่มเป็นสองเท่า การค้นหาก็ช้าลงเป็นสองเท่า

ทางแก้คือใช้ดัชนี หรือ index ซึ่งเปรียบเหมือนสารบัญท้ายหนังสือ ช่วยให้กระโดดไปหาข้อมูลได้เลยโดยไม่ต้องอ่านทุกหน้า

แต่ดัชนีมีราคาที่ต้องจ่าย ทุกครั้งที่เขียนข้อมูล ต้องอัปเดตดัชนีด้วย การเขียนจึงช้าลง นี่คือการแลกเปลี่ยนสำคัญของระบบจัดเก็บข้อมูล ดัชนีที่ดีช่วยให้อ่านเร็วขึ้น แต่ทำให้เขียนช้าลงเสมอ ฐานข้อมูลจึงไม่สร้างดัชนีให้ทุกอย่างโดยอัตโนมัติ เราต้องเลือกเองว่าจะสร้างดัชนีให้ข้อมูลส่วนไหน

ตอนที่ 3: ดัชนีแบบแฮช หรือ Hash Index

ดัชนีแบบง่ายที่สุดคือเก็บตารางในหน่วยความจำ ว่าแต่ละคีย์อยู่ตำแหน่งไหนในไฟล์ เวลาจะหาข้อมูล ก็ดูตารางนี้ แล้วกระโดดไปอ่านตำแหน่งนั้นได้ทันที ฐานข้อมูลชื่อ Bitcask ใช้วิธีนี้

วิธีนี้เหมาะกับงานที่คีย์มีไม่มาก แต่ค่าเปลี่ยนบ่อย เช่น การนับยอดวิวของวิดีโอแต่ละคลิป มีวิดีโอจำนวนจำกัด แต่ยอดวิวเพิ่มขึ้นตลอดเวลา

ปัญหาไฟล์ใหญ่ขึ้นเรื่อย ๆ

ถ้าเขียนต่อท้ายไปเรื่อย ๆ ดิสก์ก็จะเต็มในที่สุด วิธีแก้คือแบ่งไฟล์เป็นช่วง ๆ เรียกว่า segment พอไฟล์หนึ่งใหญ่ถึงขนาดที่กำหนด ก็ปิดไฟล์นั้นแล้วเริ่มไฟล์ใหม่

จากนั้นระบบจะทำงานเบื้องหลังที่เรียกว่า compaction คือการบีบอัด โดยเก็บไว้เฉพาะค่าล่าสุดของแต่ละคีย์ แล้วทิ้งค่าเก่าที่ไม่ใช้แล้ว ระบบยังรวมไฟล์เล็กหลายไฟล์เป็นไฟล์ใหม่ไฟล์เดียวได้ด้วย ระหว่างนั้นก็ยังอ่านข้อมูลจากไฟล์เก่าได้ตามปกติ พอรวมเสร็จก็เปลี่ยนไปใช้ไฟล์ใหม่ แล้วลบไฟล์เก่าทิ้ง

รายละเอียดที่ต้องคิดในระบบจริง

- การลบข้อมูล ทำโดยเขียนเครื่องหมายพิเศษว่าคีย์นี้ถูกลบแล้ว เรียกว่า tombstone หรือป้ายหลุมศพ ตอนรวมไฟล์ ระบบจะเห็นป้ายนี้แล้วทิ้งค่าเก่าของคีย์นั้นไป
- ถ้าเครื่องดับ ตารางในหน่วยความจำจะหายไป วิธีแก้คือบันทึกสำเนาของตารางลงดิสก์เป็นระยะ จะได้ไม่ต้องอ่านไฟล์ทั้งหมดใหม่ตอนเปิดเครื่อง
- ถ้าเครื่องดับตอนเขียนข้อมูลไปได้ครึ่งเดียว ระบบใช้ค่าตรวจสอบที่เรียกว่า checksum เพื่อจับข้อมูลที่เสียและข้ามไป
- ส่วนการเขียนพร้อมกันหลายคน แก้ด้วยการให้มีผู้เขียนแค่คนเดียว แต่ให้อ่านพร้อมกันได้หลายคน

ทำไมต้องเขียนต่อท้าย ไม่เขียนทับ

ฟังดูเปลืองพื้นที่ แต่มีเหตุผลดี ๆ หลายข้อ ข้อแรก การเขียนต่อเนื่องเป็นแนวยาวเร็วกว่าการกระโดดไปเขียนตามจุดต่าง ๆ มาก โดยเฉพาะบนฮาร์ดดิสก์แบบจานหมุน ข้อสอง ถ้าเครื่องดับกลางทาง ข้อมูลเก่ายังอยู่ครบ กู้คืนง่าย ข้อสาม การรวมไฟล์ช่วยลดปัญหาพื้นที่ว่างกระจัดกระจาย

ข้อจำกัด

ดัชนีแบบแฮชมีจุดอ่อนสองข้อ ข้อแรก คีย์ทั้งหมดต้องใส่ในหน่วยความจำได้ ถ้าคีย์เยอะมากก็ใช้ไม่ได้ ข้อสอง ค้นหาแบบเป็นช่วงไม่เก่ง เช่น ถ้าอยากได้ทุกคีย์ตั้งแต่หนึ่งพันถึงสองพัน ต้องค้นทีละคีย์

ตอนที่ 4: SSTable และ LSM-Tree

คราวนี้มาปรับไอเดียเดิมเล็กน้อย คือ ให้ข้อมูลในแต่ละไฟล์เรียงตามคีย์ และแต่ละคีย์ปรากฏในไฟล์เดียวเพียงครั้งเดียว ไฟล์แบบนี้เรียกว่า SSTable ย่อมาจาก Sorted String Table

การเรียงคีย์ทำให้ได้ข้อดีสามอย่าง

ข้อแรก รวมไฟล์ได้ง่ายและเร็ว แม้ไฟล์จะใหญ่กว่าหน่วยความจำ เหมือนการรวมกองไพ่ที่เรียงแล้วสองกอง หยิบใบที่เล็กกว่าจากด้านบนของแต่ละกองไปเรื่อย ๆ ก็จะได้กองใหม่ที่เรียงแล้ว

ข้อสอง ไม่ต้องเก็บดัชนีของทุกคีย์ เก็บแค่บางคีย์ห่าง ๆ ก็พอ เหมือนพจนานุกรมที่พิมพ์คำแรกของแต่ละหน้าไว้ที่หัวกระดาษ ถ้าอยากหาคำที่อยู่ระหว่างสองคำนั้น ก็เปิดไปหน้านั้นแล้วไล่หาต่อนิดเดียว ดัชนีแบบนี้เรียกว่า sparse index หรือดัชนีแบบห่าง

ข้อสาม จัดข้อมูลเป็นก้อน แล้วบีบอัดแต่ละก้อนได้ ประหยัดทั้งพื้นที่ดิสก์และการรับส่งข้อมูล

แล้วจะเรียงข้อมูลได้อย่างไร ในเมื่อข้อมูลเข้ามาแบบสุ่ม

คำตอบคือ เรียงในหน่วยความจำก่อน ขั้นตอนเป็นแบบนี้

- เมื่อมีข้อมูลเข้ามา ให้ใส่ในโครงสร้างต้นไม้ที่เรียงตัวเองอัตโนมัติในหน่วยความจำ เรียกว่า memtable
- เมื่อ memtable ใหญ่ถึงขนาดหนึ่ง เช่น ไม่กี่เมกะไบต์ ก็เขียนลงดิสก์เป็นไฟล์ SSTable ซึ่งเรียงไว้แล้วเรียบร้อย
- เวลาค้นหา ให้ดูใน memtable ก่อน ถ้าไม่เจอก็ดูไฟล์ล่าสุดบนดิสก์ แล้วไล่ไปไฟล์ที่เก่ากว่า
- เบื้องหลัง ระบบจะรวมและบีบอัดไฟล์อยู่เป็นระยะ

ปัญหาคือ ถ้าเครื่องดับ ข้อมูลใน memtable จะหายไป จึงต้องเขียนข้อมูลแบบต่อท้ายลงไฟล์บันทึกแยกต่างหากไว้ด้วยเพื่อใช้กู้คืน

โครงสร้างทั้งหมดนี้เรียกว่า LSM-Tree ย่อมาจาก Log-Structured Merge-Tree ฐานข้อมูลที่ใช้แนวคิดนี้ เช่น LevelDB, RocksDB, Cassandra, HBase และ Bigtable ของ Google ระบบค้นหาข้อความอย่าง Lucene ซึ่งเป็นหัวใจของ Elasticsearch ก็ใช้แนวคิดคล้ายกันนี้

การปรับแต่งประสิทธิภาพ

ถ้าค้นหาคีย์ที่ไม่มีอยู่จริง ระบบต้องไล่ดูทุกไฟล์จนครบ ซึ่งเสียเวลามาก จึงมีเครื่องมือชื่อ Bloom filter ซึ่งบอกได้อย่างรวดเร็วว่า คีย์นี้ไม่มีแน่นอน ระบบจะได้ไม่ต้องไปเปิดไฟล์ให้เสียเวลา

ส่วนวิธีรวมไฟล์มีสองแบบหลัก แบบแรกคือรวมไฟล์เล็กที่ใหม่กว่าเข้ากับไฟล์ใหญ่ที่เก่ากว่า เรียกว่า size-tiered แบบที่สองคือแบ่งข้อมูลเป็นชั้น ๆ แล้วค่อย ๆ ย้ายข้อมูลเก่าลงไปชั้นล่าง เรียกว่า leveled

ตอนที่ 5: B-Tree

ดัชนีที่ใช้กันแพร่หลายที่สุดคือ B-Tree เกิดขึ้นตั้งแต่ปี 1970 และยังเป็นดัชนีมาตรฐานของฐานข้อมูลเชิงสัมพันธ์แทบทุกตัวจนถึงทุกวันนี้

B-Tree ก็เรียงคีย์เหมือน SSTable แต่แนวคิดต่างกันมาก แทนที่จะแบ่งเป็นไฟล์ขนาดต่าง ๆ แล้วเขียนต่อท้าย B-Tree แบ่งข้อมูลเป็นหน้าขนาดคงที่ เรียกว่า page ปกติหน้าละประมาณ 4 กิโลไบต์ แล้วอ่านหรือเขียนทีละหน้า

B-Tree ทำงานอย่างไร

ให้นึกภาพการหาหนังสือในห้องสมุดใหญ่ ป้ายหน้าห้องบอกว่าหนังสือหมวด ก ถึง ฌ อยู่ชั้นหนึ่ง หมวด ญ ถึง ธ อยู่ชั้นสอง พอขึ้นไปชั้นสอง ก็มีป้ายบอกว่าตู้ไหนเก็บอะไร เดินไปเรื่อย ๆ จนเจอหนังสือที่ต้องการ

B-Tree ก็เป็นแบบนั้น หน้าแรกเรียกว่า root หรือราก ในหน้านี้มีคีย์ไม่กี่ตัว และตัวชี้ไปยังหน้าลูกที่ดูแลช่วงคีย์ต่าง ๆ ไล่ลงไปเรื่อย ๆ จนถึงหน้าล่างสุด ซึ่งเก็บข้อมูลจริง

แต่ละหน้าชี้ไปหน้าลูกได้หลายร้อยหน้า ต้นไม้จึงไม่สูงมาก ฐานข้อมูลส่วนใหญ่ใช้แค่สามหรือสี่ชั้น ตัวอย่างในหนังสือบอกว่า ถ้าหน้าละ 4 กิโลไบต์ ชี้ไปหน้าลูกได้ 500 หน้า แค่สี่ชั้นก็เก็บข้อมูลได้ถึงประมาณ 256 เทราไบต์

การแก้ไขและเพิ่มข้อมูล

ต่างจาก LSM-Tree ตรงที่ B-Tree เขียนทับข้อมูลเดิมในหน้าเดิมเลย ถ้าเพิ่มคีย์ใหม่ แล้วหน้านั้นเต็ม ก็แบ่งหน้าเป็นสองหน้า แล้วไปแก้หน้าพ่อให้ชี้ไปยังทั้งสองหน้า

ความเสี่ยงของการเขียนทับ

การเขียนทับเสี่ยงกว่าการเขียนต่อท้าย เช่น ตอนแบ่งหน้า ต้องเขียนหลายหน้าพร้อมกัน ถ้าเครื่องดับกลางทาง ต้นไม้อาจเสียหายได้ เช่น มีหน้าที่ไม่มีใครชี้ถึงเลย

วิธีป้องกันคือ ก่อนแก้ไขอะไรใน B-Tree ให้บันทึกสิ่งที่จะทำลงไฟล์บันทึกแบบต่อท้ายก่อน ไฟล์นี้เรียกว่า write-ahead log หรือ WAL ถ้าเครื่องดับ ก็ใช้ไฟล์นี้กู้ต้นไม้กลับมาให้ถูกต้อง

นอกจากนี้ ถ้ามีหลายคนแก้ข้อมูลพร้อมกัน ต้องมีการล็อกแบบเบา ๆ ที่เรียกว่า latch เพื่อไม่ให้ใครเห็นต้นไม้ในสภาพที่แก้ไปได้ครึ่งเดียว

การปรับปรุงเพิ่มเติม

B-Tree มีการพัฒนามาหลายสิบปี จึงมีเทคนิคเสริมมากมาย เช่น

- แทนที่จะเขียนทับ ให้เขียนหน้าใหม่แล้วเปลี่ยนตัวชี้ เรียกว่า copy-on-write ฐานข้อมูล LMDB ใช้วิธีนี้
- ย่อคีย์ในหน้าชั้นบนให้สั้นลง เพื่อให้ใส่ได้มากขึ้นต่อหน้า และต้นไม้จะได้เตี้ยลง
- เพิ่มตัวชี้ไปยังหน้าข้าง ๆ เพื่อให้ไล่อ่านข้อมูลตามลำดับได้โดยไม่ต้องย้อนกลับขึ้นไปหน้าพ่อ

ตอนที่ 6: LSM-Tree เทียบกับ B-Tree

หลักง่าย ๆ คือ LSM-Tree มักเขียนเร็วกว่า ส่วน B-Tree มักอ่านเร็วกว่า แต่ผลจริงขึ้นอยู่กับงานแต่ละแบบ ควรทดสอบกับงานของตัวเองเสมอ

ข้อดีของ LSM-Tree

ข้อแรก มีปัญหาเรื่องการเขียนซ้ำซ้อนน้อยกว่า ข้อมูลหนึ่งชิ้นอาจถูกเขียนลงดิสก์หลายรอบ เช่น เขียนลงไฟล์บันทึก เขียนลงตัวโครงสร้าง แล้วเขียนอีกรอบตอนรวมไฟล์ ปรากฏการณ์นี้เรียกว่า write amplification B-Tree มักเจอปัญหานี้หนักกว่า เพราะแก้แค่ไม่กี่ไบต์ก็ต้องเขียนทั้งหน้า ซึ่งเป็นเรื่องสำคัญโดยเฉพาะกับ SSD ที่เขียนได้จำนวนครั้งจำกัด

ข้อสอง ข้อมูลอัดแน่นกว่า กินพื้นที่น้อยกว่า เพราะ B-Tree มักมีที่ว่างเหลือในแต่ละหน้า

ข้อเสียของ LSM-Tree

ข้อแรก การรวมไฟล์เบื้องหลังอาจไปแย่งทรัพยากรดิสก์กับงานหลัก ทำให้บางคำขอต้องรอนานผิดปกติ เวลาตอบสนองโดยเฉลี่ยอาจดี แต่ค่าเปอร์เซ็นไทล์สูง ๆ ที่เราพูดถึงในบทที่หนึ่งอาจแย่ลง

ข้อสอง ถ้ามีข้อมูลเขียนเข้ามาเร็วมาก การรวมไฟล์อาจตามไม่ทัน ไฟล์จะสะสมจนดิสก์เต็ม และการอ่านช้าลงเรื่อย ๆ ต้องมีการเฝ้าระวังเรื่องนี้ให้ดี

ข้อสาม ใน B-Tree แต่ละคีย์มีอยู่ที่เดียวเท่านั้น ส่วน LSM-Tree อาจมีหลายสำเนาในหลายไฟล์ B-Tree จึงล็อกข้อมูลเป็นช่วงได้ง่ายกว่า ซึ่งสำคัญมากสำหรับงานธุรกรรม เราจะพูดถึงเรื่องนี้อีกครั้งในบทที่เจ็ด

ตอนที่ 7: ดัชนีแบบอื่น ๆ

ดัชนีรอง หรือ Secondary Index

ที่ผ่านมาเราพูดถึงดัชนีของคีย์หลัก แต่เรามักต้องการค้นจากข้อมูลอื่นด้วย เช่น ค้นหาคำสั่งซื้อทั้งหมดของลูกค้าคนหนึ่ง ดัชนีรองต่างจากดัชนีหลักตรงที่คีย์ซ้ำกันได้ เพราะลูกค้าคนเดียวอาจมีคำสั่งซื้อหลายรายการ

จะเก็บข้อมูลจริงไว้ตรงไหน

ทางแรก ให้ดัชนีเก็บแค่ตัวชี้ไปยังข้อมูล ซึ่งอยู่ในพื้นที่แยกต่างหากที่เรียกว่า heap file วิธีนี้ไม่ต้องเก็บข้อมูลซ้ำหลายที่

ทางที่สอง เก็บข้อมูลทั้งแถวไว้ในดัชนีเลย เรียกว่า clustered index อ่านได้เร็ว เพราะไม่ต้องกระโดดไปอีกที่ เช่น ใน MySQL ที่ใช้ InnoDB คีย์หลักจะเป็นดัชนีแบบนี้เสมอ

ทางที่สาม อยู่ตรงกลาง คือเก็บแค่บางคอลัมน์ไว้ในดัชนี เรียกว่า covering index ถ้าคำสั่งค้นหาใช้แค่คอลัมน์ที่อยู่ในดัชนี ก็ตอบได้ทันทีโดยไม่ต้องไปอ่านตารางจริง

ข้อแลกเปลี่ยนคือ ยิ่งเก็บข้อมูลซ้ำในดัชนีมาก อ่านก็ยิ่งเร็ว แต่ใช้พื้นที่มากขึ้น และเขียนช้าลง

ดัชนีหลายคอลัมน์

แบบที่พบบ่อยคือเอาหลายคอลัมน์มาต่อกันเป็นคีย์เดียว เหมือนสมุดโทรศัพท์ที่เรียงตามนามสกุลก่อน แล้วค่อยเรียงตามชื่อ ใช้หาจากนามสกุลอย่างเดียวได้ หรือหาจากนามสกุลพร้อมชื่อได้ แต่ถ้าหาจากชื่ออย่างเดียว ดัชนีนี้ช่วยไม่ได้

อีกแบบคือดัชนีหลายมิติ ซึ่งสำคัญมากกับข้อมูลแผนที่ เช่น ถ้าอยากหาร้านอาหารทั้งหมดในกรอบสี่เหลี่ยมบนแผนที่ ต้องค้นทั้งละติจูดและลองจิจูดพร้อมกัน ดัชนีธรรมดาทำได้ไม่ดี จึงต้องใช้ดัชนีเฉพาะทาง เช่น R-Tree ซึ่ง PostGIS ใช้

การค้นหาแบบคลุมเครือ

บางครั้งเราอยากค้นหาแม้สะกดผิด หรือค้นหาคำที่มีความหมายใกล้เคียงกัน ระบบค้นหาข้อความอย่าง Lucene รองรับการค้นหาคำที่สะกดต่างกันไม่เกินจำนวนตัวอักษรที่กำหนด ซึ่งดัชนีทั่วไปทำไม่ได้

ฐานข้อมูลในหน่วยความจำ

เมื่อหน่วยความจำราคาถูกลง ข้อมูลหลายชุดก็ใส่ในหน่วยความจำได้ทั้งหมด จึงเกิดฐานข้อมูลอย่าง Redis, Memcached และ VoltDB

ผู้เขียนชี้จุดที่หลายคนเข้าใจผิด ฐานข้อมูลเหล่านี้ไม่ได้เร็วเพราะไม่ต้องอ่านดิสก์ เพราะฐานข้อมูลบนดิสก์ก็มักมีข้อมูลที่ใช้บ่อยค้างอยู่ในหน่วยความจำอยู่แล้ว เหตุผลจริงที่เร็วกว่า คือไม่ต้องเสียเวลาแปลงข้อมูลให้อยู่ในรูปแบบที่เขียนลงดิสก์ได้

ข้อดีอีกอย่างคือ ใช้โครงสร้างข้อมูลที่ทำบนดิสก์ได้ยาก เช่น Redis มีโครงสร้างอย่างคิวและเซตให้ใช้ได้ตรง ๆ

ตอนที่ 8: งานธุรกรรม กับ งานวิเคราะห์ข้อมูล

ทีนี้มาถึงส่วนที่สองของบท

การใช้ฐานข้อมูลแบ่งได้เป็นสองแบบใหญ่ ๆ

แบบแรกเรียกว่า OLTP ย่อมาจาก Online Transaction Processing คืองานธุรกรรมทั่วไป เช่น ลูกค้ากดสั่งซื้อสินค้า หรือแก้ที่อยู่จัดส่ง งานแบบนี้อ่านหรือเขียนข้อมูลทีละไม่กี่แถว แต่มีคำขอเข้ามาจำนวนมาก และต้องตอบเร็ว

แบบที่สองเรียกว่า OLAP ย่อมาจาก Online Analytic Processing คืองานวิเคราะห์ข้อมูล เช่น ผู้บริหารอยากรู้ว่าเดือนที่แล้ว แต่ละสาขามียอดขายรวมเท่าไร งานแบบนี้ต้องอ่านข้อมูลหลายล้านแถว แต่ใช้แค่ไม่กี่คอลัมน์ และผลลัพธ์มักเป็นตัวเลขสรุป

คลังข้อมูล หรือ Data Warehouse

บริษัทไม่อยากให้นักวิเคราะห์ไปรันคำสั่งหนัก ๆ บนฐานข้อมูลหลักที่ลูกค้าใช้อยู่ เพราะจะทำให้ระบบช้าลง จึงคัดลอกข้อมูลจากหลายระบบมาไว้ในฐานข้อมูลอีกตัวหนึ่ง เรียกว่าคลังข้อมูล

ขั้นตอนการคัดลอกข้อมูลนี้เรียกว่า ETL ย่อมาจาก Extract, Transform, Load คือ ดึงข้อมูลออกมา แปลงให้อยู่ในรูปแบบที่วิเคราะห์ง่าย แล้วโหลดเข้าคลังข้อมูล

คลังข้อมูลมักใช้ภาษา SQL เหมือนกัน แต่ข้างในออกแบบมาต่างกันโดยสิ้นเชิง เพื่อให้เหมาะกับงานวิเคราะห์ ตัวอย่างระบบ เช่น Amazon Redshift, Vertica และ Teradata ส่วนฝั่งโอเพนซอร์ส เช่น Hive, Spark SQL และ Presto

โครงสร้างแบบดาว หรือ Star Schema

คลังข้อมูลส่วนใหญ่จัดข้อมูลเป็นรูปดาว ตรงกลางคือตารางข้อเท็จจริง หรือ fact table ซึ่งเก็บเหตุการณ์ เช่น การขายแต่ละครั้ง แต่ละแถวบอกว่าขายอะไร ที่สาขาไหน ราคาเท่าไร

รอบ ๆ ตารางกลาง มีตารางมิติ หรือ dimension table หลายตาราง ซึ่งเก็บรายละเอียดเพิ่มเติม เช่น ตารางสินค้า ตารางสาขา ตารางลูกค้า และตารางวันที่ เมื่อวาดเป็นแผนภาพ ตารางกลางเชื่อมไปหาตารางรอบ ๆ ดูเหมือนดาว จึงเป็นที่มาของชื่อ

ถ้าแตกตารางมิติออกไปอีกชั้น เช่น ตารางสินค้าชี้ไปหาตารางยี่ห้อ ก็จะเรียกว่าโครงสร้างแบบเกล็ดหิมะ หรือ snowflake schema

ตารางข้อเท็จจริงมักใหญ่มาก อาจมีหลายร้อยคอลัมน์ และข้อมูลระดับหลายเพตะไบต์

ตอนที่ 9: การเก็บข้อมูลแบบคอลัมน์

ลองนึกว่าตารางยอดขายมีหลายร้อยคอลัมน์ แต่คำสั่งวิเคราะห์ส่วนใหญ่ใช้แค่สี่หรือห้าคอลัมน์

ฐานข้อมูลทั่วไปเก็บข้อมูลทีละแถว ข้อมูลทุกคอลัมน์ของแถวเดียวกันอยู่ติดกัน เวลาจะอ่านแค่ห้าคอลัมน์ ก็ต้องอ่านทั้งแถว ซึ่งเปลืองมาก

ทางแก้คือเก็บข้อมูลทีละคอลัมน์ แต่ละคอลัมน์อยู่ในไฟล์ของตัวเอง เวลาค้นหาก็อ่านเฉพาะคอลัมน์ที่ต้องใช้ เรียกว่า column-oriented storage

การบีบอัดข้อมูล

ข้อมูลในคอลัมน์เดียวกันมักซ้ำกันเยอะ เช่น คอลัมน์รหัสสาขา อาจมีแค่ร้อยค่า แต่มีหลายพันล้านแถว จึงบีบอัดได้ดีมาก

เทคนิคหนึ่งคือ bitmap encoding คือสร้างแถวของเลขศูนย์กับหนึ่งให้แต่ละค่า ว่าแถวไหนมีค่านี้ แล้วบีบอัดต่อด้วยการนับจำนวนที่ซ้ำกันต่อเนื่อง เช่น แทนที่จะเขียนศูนย์หนึ่งร้อยตัว ก็เขียนว่า ศูนย์ หนึ่งร้อยตัว แทน เรียกว่า run-length encoding

นอกจากประหยัดพื้นที่แล้ว ข้อมูลที่บีบอัดแน่น ๆ ยังใส่ในหน่วยความจำแคชของซีพียูได้พอดี ซีพียูจึงประมวลผลข้อมูลทีละก้อนได้เร็วมาก เทคนิคนี้เรียกว่า vectorized processing

การเรียงลำดับข้อมูล

ถ้าเรียงแถวตามคอลัมน์ที่ค้นบ่อย เช่น เรียงตามวันที่ การค้นหาข้อมูลเดือนที่แล้วจะเร็วมาก แถมคอลัมน์ที่เรียงแล้วยังมีค่าซ้ำต่อเนื่องกันยาว ๆ จึงบีบอัดได้ดีขึ้นอีก

มีไอเดียที่ฉลาดจากฐานข้อมูล Vertica ด้วย คือในเมื่อต้องเก็บสำเนาข้อมูลหลายชุดไว้กันข้อมูลหายอยู่แล้ว ก็ให้แต่ละสำเนาเรียงต่างกันไปเลย เวลาค้นหาก็เลือกสำเนาที่เรียงเหมาะกับคำสั่งนั้นที่สุด

การเขียนข้อมูล

การเก็บแบบคอลัมน์มีข้อเสียคือเขียนข้อมูลยาก ถ้าจะแทรกแถวใหม่ตรงกลาง ต้องแก้ทุกไฟล์คอลัมน์ วิธีแก้คือใช้แนวคิดเดียวกับ LSM-Tree คือเขียนลงหน่วยความจำก่อน แล้วค่อยรวมลงไฟล์คอลัมน์บนดิสก์เป็นระยะ

การคำนวณผลรวมไว้ล่วงหน้า

งานวิเคราะห์มักใช้ผลรวม ค่าเฉลี่ย หรือจำนวนนับซ้ำ ๆ จึงคำนวณเก็บไว้ล่วงหน้าได้ เรียกว่า materialized view

แบบที่พบบ่อยคือ data cube เช่น ตารางที่แกนหนึ่งเป็นวันที่ อีกแกนเป็นสินค้า แต่ละช่องเก็บยอดขายรวม ถ้าอยากรู้ยอดขายของสินค้าหนึ่งตลอดทั้งปี ก็แค่อ่านผลรวมที่คำนวณไว้แล้ว ไม่ต้องอ่านข้อมูลดิบหลายล้านแถว ข้อเสียคือยืดหยุ่นน้อย ถ้าอยากถามคำถามที่ไม่ได้เตรียมไว้ ก็ต้องกลับไปใช้ข้อมูลดิบอยู่ดี

ตอนที่ 10: สรุปท้ายบท

มาทบทวนกันอีกครั้งครับ

หนึ่ง ระบบจัดเก็บข้อมูลแบ่งได้สองกลุ่ม กลุ่มแรกสำหรับงานธุรกรรม ซึ่งมีคำขอเล็ก ๆ จำนวนมาก ปัญหาหลักคือการหาข้อมูลบนดิสก์ให้เร็ว กลุ่มที่สองสำหรับงานวิเคราะห์ ซึ่งมีคำขอน้อยแต่หนักมาก ปัญหาหลักคือแบนด์วิดท์ของดิสก์

สอง ฝั่งงานธุรกรรมมีสองแนวคิดหลัก แนวคิดแรกคือแบบเขียนต่อท้าย เช่น LSM-Tree ซึ่งเปลี่ยนการเขียนแบบกระจัดกระจายให้เป็นการเขียนต่อเนื่อง จึงเขียนได้เร็ว แนวคิดที่สองคือแบบเขียนทับ เช่น B-Tree ซึ่งเป็นดัชนีมาตรฐานของฐานข้อมูลเชิงสัมพันธ์มาหลายสิบปี

สาม ทุกดัชนีมีราคา คือช่วยให้อ่านเร็วขึ้น แต่ทำให้เขียนช้าลง ต้องเลือกให้เหมาะกับงาน

สี่ ฝั่งงานวิเคราะห์ การเก็บข้อมูลแบบคอลัมน์ช่วยให้อ่านเฉพาะข้อมูลที่จำเป็น และบีบอัดได้ดี จึงเหมาะกับการสแกนข้อมูลหลายล้านแถว

ห้า เมื่อเข้าใจว่าฐานข้อมูลทำงานอย่างไรข้างใน เราจะเลือกเครื่องมือได้ถูกกับงาน และรู้ว่าควรปรับแต่งตรงไหน

บทต่อไปคือบทที่สี่ ว่าด้วยการเข้ารหัสข้อมูลและการพัฒนาระบบ เช่น JSON, Protocol Buffers และการทำให้ระบบเก่ากับระบบใหม่คุยกันรู้เรื่อง ขอบคุณที่ฟังครับ

ทิปสำหรับ Edge: บทนี้ยาวกว่าสองบทแรก ถ้าฟังรวดเดียวแล้วรู้สึกหนัก แนะนำให้แบ่งฟังเป็นสองรอบ รอบแรกตอนที่ 1–7 (งานธุรกรรม) และรอบสองตอนที่ 8–10 (งานวิเคราะห์) ครับ

จะไปต่อบทที่ 4 เลยไหมครับ หรือจะให้รวมบทที่ 1–3 เป็นไฟล์ PDF ไว้เปิดฟังใน Edge ก็ได้ 😊